Lab: Study Tracker

Your first Flutter app: widgets, StatelessWidget, StatefulWidget, setState and the State lifecycle

Duration	90 minutes
Format	Individual work. Every student has personalised numbers.
Slides	Session 2, slides 18 to 38 (Meet Flutter through the State lifecycle)
Tools	Flutter SDK (`flutter run`, `flutter analyze`), VS Code, Git, your existing course repository on GitHub
Submission	`git push` to your course repo after every step (10 pushes), plus a final push with answers
Total marks	100

Learning Objectives

By the end of this lab you will be able to:

- Explain how a Flutter app starts: `main`, `runApp`, `MaterialApp`, `Scaffold`
- Write a `StatelessWidget` with `super.key`, a `const` constructor and `final` fields
- Build a widget tree with `Column`, `Row`, `Text` and buttons, and read it as a tree
- Write a `StatefulWidget` as two classes and change the screen with `setState`
- Find and fix the classic "forgot `setState`" bug
- Use `initState` and `dispose` correctly with a `Timer`, and count how often `build` runs

Your Personal Version of This Lab

Every student gets **different numbers**, so every student's app behaves differently. The starter code reads the **last two digits of your roll number** and turns them into a seed:

- seed is the two-digit number (00 to 99). For roll number 21F-0347 the seed is 47.
- t is the tens digit (4) and u is the units digit (7).

Your course name, the starting value, the step size, the maximum and the countdown start are all calculated from t and u. An app that is correct for another roll number will **not** match yours, and that is intended.

Rules: do not edit the seeded settings in the starter file; never use `t` or `u` as names of your own variables; use whole-number (int) arithmetic only, and `~/` for division; keep every `Key(...)` and every text format exactly as shown. Your instructor checks your app with an automatic test generated for your own roll number, so a text that is one character different counts as wrong.

Time Plan and Checkpoints

Each step ends with a **checkpoint**: you commit and push. The "finish by" column is the minute (from the start of the lab) by which your push should be on GitHub. GitHub records when each push arrived.

Checkpoint	Topic	Time	Finish by
Setup	Create the project, first push	8 min	min 8
Step 1	`main`, `runApp`, `MaterialApp`, `Scaffold`	5 min	min 13
Step 2	`StatelessWidget` and `super.key`	7 min	min 20
Step 3	The widget tree: `Column` and `Row`	7 min	min 27
Step 4	`build()` describes the screen	6 min	min 33
Step 5	`StatefulWidget`: two classes	8 min	min 41
Step 6	`setState` and its limits	8 min	min 49
Step 7	The classic bug: no `setState`	6 min	min 55
Step 8	Lifecycle: `initState`, `Timer`, `dispose`	9 min	min 64
Step 9	Counting `build` calls, the lifecycle log	6 min	min 70
Step 10	Capstone: the whole screen	5 min	min 75
Wrap-up	Analyse, answers, screenshot, final push	15 min	min 90
Total		**90 min**	

How to submit: your GitHub repository

You already have a repository for this course. Do **not** create a new one. Put everything for this lab in a Flutter project named `flutter_lab` at the top level of that repository:

```
your-course-repo/
  flutter_lab/
    lib/main.dart          <- your program (one file)
    ANSWERS.md             <- numeric and written answers (at the end)
    screenshot.png         <- a screenshot of your running app (at the end)
```

At the end of **every step**, run these commands from the top of your repository (replace N):

```
git add flutter_lab
git commit -m "Flutter lab: step N"
git push
```

Integrity rules: never force-push, rebase or amend a commit that you already pushed. Your commit history is your proof that you did the work in order and in time. Commit messages must follow the format above.

Setup (8 min)

- Open a terminal in your course repository and run `git pull` so you start up to date.
- Run `flutter create flutter_lab`. Open the new folder in VS Code.
- Delete the file `flutter_lab/test/widget_test.dart` (it tests the counter app that you are about to replace).
- Replace the whole content of `flutter_lab/lib/main.dart` with the starter code below (your instructor also shares it as `flutter_lab_starter.dart`).
- Type **your name** and **your roll number**, which must end in two digits.

- Run the app (`flutter run -d chrome`, or an emulator or your phone). You should see the text `Setup OK: seed ...`
- Commit with the message `Flutter lab: setup` and push.

```
// main.dart - Study Tracker lab
import 'dart:async';

import 'package:flutter/material.dart';

const String studentName = 'YOUR NAME HERE';
const String rollNo = 'YOUR-ROLL-NO-HERE'; // e.g. '21F-0347'

// ===== Seeded settings (generated from YOUR roll number). Do not edit. =====
final int seed = int.parse(rollNo.substring(rollNo.length - 2));
final int t = seed ~/ 10; // tens digit
final int u = seed % 10; // units digit

const List<String> courses = [
  'Mobile Dev', 'Data Structures', 'Operating Systems', 'Databases',
  'Networks', 'Software Eng', 'AI', 'Web Engineering', 'Graphics', 'Security',
];

final String course = courses[u]; // your course name
final int startCount = 3 + u; // where the tally starts
final int stepSize = 1 + t % 4; // how much + and - change the tally
final int maxCount = 20 + 2 * t; // the tally may never go above this
final int startSeconds = 5 + (t + u) % 6; // where the countdown starts
final int bugStep = 1 + u % 4; // used by BuggyCounter in Step 7
// ============================================================================

// Step 9 writes to this list from initState and dispose.
final List<String> lifecycleLog = <String>[];

void main() => runApp(const LabApp());

// Keep the name LabApp and its const constructor: the checker uses them.
class LabApp extends StatelessWidget {
  const LabApp({super.key});

  @override
  Widget build(BuildContext context) {
    // Step 1 replaces everything below.
    return MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Setup OK: seed $seed (t=$t, u=$u)')),
      ),
    );
  }
}

// ---- Provided for Step 7. It has a bug on purpose. Do not rename anything.
class BuggyCounter extends StatefulWidget {
  const BuggyCounter({super.key});

  @override
  State<BuggyCounter> createState() => _BuggyCounterState();
}

class _BuggyCounterState extends State<BuggyCounter> {
```

```
  int _clicks = 0;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text('Bug clicks: $_clicks', key: const Key('bug-text')),
        const SizedBox(width: 12),
        ElevatedButton(
          key: const Key('bug-button'),
          onPressed: () {
            _clicks += bugStep;
          },
          child: Text('+$bugStep'),
        ),
      ],
    );
  }
}
```

If you get a FormatException straight away, you have not replaced the placeholder roll number, or it does not end in two digits. Write all your widgets in this one file, below the seeded settings. Keep the names given in each step, because the checker looks for them. Every widget that has a **Key** below must have exactly that key, for example `Text(..., key: const Key('count'))`.

Step 1: How an App Starts (5 min)

Slides 26 and 27. main() calls runApp(); MaterialApp is the root; Scaffold is the page.

Task 1.1

In `LabApp.build` return a `MaterialApp` with `title: 'Study Tracker'` and `home: const HomePage()`. Because `HomePage` is const, `LabApp.build` can now return `const MaterialApp(...)`.

Task 1.2

Write a new class `HomePage extends StatelessWidget` (with `const HomePage({super.key});`). Its `build` returns a `Scaffold` whose `appBar` is `AppBar(title: Text('Study Tracker: $course'))` and whose `body` is `const Center(child: Text('Hello'))` for now.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
AppBar title:   Study Tracker: <your course>      (course comes from the seeded settings)
```

Self-check: the AppBar shows your own course, not Mobile Dev unless your units digit is 0.

Think (answer in a code comment): What is the difference between what `runApp` receives and what `MaterialApp` receives as `home`?

PUSH. Checkpoint 1: commit message `Flutter lab: step 1`, then `git push`.

Step 2: StatelessWidget and super.key (7 min)

Slide 31 and the StatelessWidget anatomy slide. A widget whose output depends only on what it was given.

Task 2.1

Write `class StudentCard extends StatelessWidget` with three **final** String fields `name`, `roll` and `course`, and a **const** constructor with `super.key` and three `required` named parameters:

```
const StudentCard({
  super.key,
  required this.name,
  required this.roll,
  required this.course,
});
```

Task 2.2

Its `build` returns a `Card` containing a `Padding` (16 on all sides) containing a `Column` with `crossAxisAlignment: CrossAxisAlignment.start` and three `Text` widgets:

```
Text('Name: $name', key: const Key('student-name'))
Text('Roll no: $roll', key: const Key('student-roll'))
Text('Course: $course', key: const Key('student-course'))
```

Task 2.3

Use it in `HomePage` instead of the `Center` text: `StudentCard(name: studentName, roll: rollNo, course: course)`. Put it as the `body` for now.

Self-check: the card shows your name, your full roll number and your course on three lines.

Think (answer in a code comment): Why are the fields of a StatelessWidget `final`? What would the compiler say if you removed `final`?

PUSH. Checkpoint 2: commit message `Flutter lab: step 2`, then `git push`.

Step 3: The Widget Tree: Column and Row (7 min)

Slide 28. Everything is a widget; layouts are widgets that hold other widgets.

Task 3.1

Change the `body` of `HomePage` to this outline. Keep the `SingleChildScrollView`: it lets the page scroll instead of overflowing on a small screen.

```
body: SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      StudentCard(...),            // from Step 2
      const SizedBox(height: 12),
      // TODO: the info Row
    ],
  ),
),
```

Task 3.2

Add a `Row` with `mainAxisAlignment: MainAxisAlignment.spaceBetween` and three `Text` children that show the seeded values:

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Start: $startCount', key: const Key('info-start'))
Text('Step: $stepSize',    key: const Key('info-step'))
Text('Max: $maxCount',     key: const Key('info-max'))
```

Self-check: the three info texts spread across the width, with the first at the left edge and the last at the right edge.

Think (answer in a code comment): Draw the widget tree of your screen on paper, from `LabApp` down to the three Text widgets. How many levels deep is the `Text('Max: ..')`?

PUSH. Checkpoint 3: commit message `Flutter lab: step 3`, then `git push`.

Step 4: build() Describes the Screen (6 min)

Slide 29. build() takes data and returns widgets. Same input, same picture.

Task 4.1

Write `class GoalBanner extends StatelessWidget` with two `final int` fields, `count` and `max`, and a const constructor `const GoalBanner({super.key, required this.count, required this.max});`.

Task 4.2

In `build`, compute the percentage with **integer division**, `final percent = count * 100 ~/ max;`, and return:

```
Text('Progress: $count of $max ($percent%)', key: const Key('progress'))
```

Task 4.3

Add `GoalBanner(count: startCount, max: maxCount)` to the Column, below the info Row (with a `SizedBox(height: 12)` before it).

Self-check: at the start the banner shows your start value, your maximum, and the percentage rounded down.

Think (answer in a code comment): Why does `GoalBanner` not need `setState`? What would you have to do to make it show a new count?

PUSH. Checkpoint 4: commit message `Flutter lab: step 4`, then `git push`.

Step 5: StatefulWidget: Two Classes (8 min)

Slide 32. The widget is the configuration; the State object is the memory.

Task 5.1

Write `class Tally extends StatefulWidget` (const constructor with `super.key`) whose `createState()` returns `_TallyState()`. Write `class _TallyState extends State<Tally>` with a field `int _count = startCount;`.

Task 5.2

Its `build` returns a `Column` (`crossAxisAlignment: CrossAxisAlignment.start`) with three children, in this order:

- `Text('Count: $_count', key: const Key('count'))`
- `GoalBanner(count: _count, max: maxCount)` (move it here from `HomePage`: **delete the old one** from the page so the key `progress` exists only once)
- a `Row` with an `ElevatedButton(key: const Key('plus'), onPressed: ..., child: Text('+$stepSize'))`

Task 5.3

Write the plus action as a method `void _plus()` that adds `stepSize` to `_count` **inside `setState`**. Pass it as `onPressed: _plus`. Put `const Tally()` in `HomePage` below the info Row.

Self-check: tapping the button changes both the count and the progress line. The progress line works because `Tally` rebuilds its child `GoalBanner` with the new count.

Think (answer in a code comment): The class `Tally` has no fields that change, yet `_TallyState` does. Why are there two classes?

PUSH. Checkpoint 5: commit message `Flutter lab: step 5`, then `git push`.

Step 6: setState and Its Limits (8 min)

Slides 33 and 34. setState runs your code, then tells Flutter to call build again.

Task 6.1

Make `_plus` stop at the maximum: after adding, `if (_count > maxCount) _count = maxCount;` (inside the same `setState`).

Task 6.2

Add `void _minus()` that subtracts `stepSize` and never goes below 0, and `void _reset()` that sets `_count` back to `startCount`. Add two buttons in the same Row, with a gap `SizedBox(width: 8)` between buttons:

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
ElevatedButton(key: const Key('minus'), ..., child: Text('-$stepSize'))
OutlinedButton(key: const Key('reset'), ..., child: const Text('Reset'))
```

Self-check: keep tapping + and the count stops at your maximum; keep tapping - and it stops at 0; Reset returns to your start value; the progress percentage follows every change.

Think (answer in a code comment): Hot reload while the count is 12. Does it stay 12? Now do a hot restart. Explain the difference using the words widget and State.

PUSH. Checkpoint 6: commit message `Flutter lab: step 6`, then `git push`.

Step 7: The Classic Bug: No setState (6 min)

Slide 34. Changing a variable is not enough: Flutter must be told to rebuild.

Task 7.1

The starter file already contains a widget `BuggyCounter`. Add `const BuggyCounter()` to the bottom of the Column in `HomePage`, with a `SizedBox(height: 12)` before it. Run the app and tap its button several times. Nothing changes on the screen.

Task 7.2

Fix it: wrap the line `_clicks += bugStep;` in `setState(() { ... });`. Do **not** rename anything in that widget.

Self-check: after the fix, each tap adds your bugStep to the number on the screen.

Think (answer in a code comment): Before the fix, you tapped three times. What was the value of `_clicks` inside the State, and what did the screen show? Write both numbers (you need the first for ANSWERS.md, A3).

PUSH. Checkpoint 7: commit message `Flutter lab: step 7`, then `git push`.

Step 8: Lifecycle: initState, Timer, dispose (9 min)

Slides 35 to 38. The State is created once, built many times, and disposed once.

Task 8.1

Write `class Countdown extends StatefulWidget` and its State. The State has `late int _left;` and `late Timer _timer;`. (`Timer` needs `import 'dart:async';`, already at the top of the starter.)

Task 8.2

In `initState` (call `super.initState()` first): set `_left = startSeconds;` and start a periodic timer:

```
_timer = Timer.periodic(const Duration(seconds: 1), (timer) {
  setState(() => _left--);
  if (_left <= 0) timer.cancel();   // stop at zero
});
```

Task 8.3

In `dispose` cancel the timer **before** `super.dispose()`: `_timer.cancel();`.

Task 8.4

`build` shows one Text. While `_left > 0` it says `Time left: <n> s`, and at zero it says `Time is up!`. Add `const Countdown()` to the Column below the Tally.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Time left: $_left s', key: const Key('time-left'))   // while _left > 0
Text('Time is up!',         key: const Key('time-left'))   // at zero: same key
```

Self-check: the number goes down once per second, stops at zero and shows Time is up!.

Think (answer in a code comment): What error appears in the console if you delete the `_timer.cancel()` from `dispose` and the widget is removed? (Try it in a small experiment by wrapping `Countdown` in an `if` controlled by a button, or just reason about it.)

PUSH. Checkpoint 8: commit message `Flutter lab: step 8`, then `git push`.

Step 9: Counting build Calls and the Lifecycle Log (6 min)

Slide 35. initState once, build many times, dispose once. Now measure it.

Task 9.1

In `_CountdownState` add `int _builds = 0;`. At the **first line** of `build` write `_builds++;`. Make `build` return a `Column` (`crossAxisAlignment: CrossAxisAlignment.start`) with your countdown Text and a second Text:

Task 9.2

Record the lifecycle in the global list given in the starter: the first line after `super.initState()` is `lifecycleLog.add('initState');`, and in `dispose` (before `super.dispose()`) `lifecycleLog.add('dispose');`.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Builds: $_builds', key: const Key('builds'))
```

Self-check: the Builds number goes up by exactly one every second until the countdown reaches zero, then stops.

Think (answer in a code comment): Why does it increase by one per second and not by two? Where could you print `lifecycleLog` to prove that `dispose` ran?

PUSH. Checkpoint 9: commit message `Flutter lab: step 9`, then `git push`.

Step 10: Capstone: The Whole Screen (5 min)

Everything so far in one app: Stateless and Stateful widgets together.

Task 10.1

Check that `HomePage` shows, from top to bottom: StudentCard, the info Row, Tally (with its GoalBanner), Countdown (with Builds), BuggyCounter. Every part must appear **once**.

Task 10.2

Add `const` wherever the analyzer suggests it (`flutter analyze` shows `prefer_const_constructors`). Make sure the page shows no yellow-black overflow stripes on a small window.

Task 10.3

Write one comment above each of your widget classes saying `// Stateless` or `// Stateful`, and one line why.

Self-check: the screen scrolls, nothing overflows, and the analyzer reports no issues.

PUSH. Checkpoint 10: commit message `Flutter lab: step 10`, then `git push`.

Wrap-up (15 min)

- Run `dart format flutter_lab/lib` and `flutter analyze` (inside `flutter_lab`). Fix every warning.
- Take a screenshot of your running app and save it as `flutter_lab/screenshot.png`. The AppBar title and your roll number must be visible.
- Create `flutter_lab/ANSWERS.md` with your name and roll number, the **four numbers** below, and short written answers:

```
A1: <how many taps of + take the tally from startCount up to maxCount>
A2: <how many times Countdown.build runs from the first display until Time is
up! is shown>
A3: <value of _clicks inside BuggyCounter after 3 taps, with the bug present>
A4: <the percent shown in the Progress line when the app starts>

B1: which of your classes are Stateless and which are Stateful, and why
B2: why must the Timer be cancelled in dispose()
B3: which widgets in your code can be const, and what const buys you
```

Write each A answer as a plain whole number (for example `A1: 7`). Your instructor checks A1 to A4 automatically against your roll number. Work them out from your seeded settings and from what you observed in the app. Do not guess: they follow from the numbers in your starter file.

- Final push, commit message `Flutter lab: final`:

```
git add flutter_lab
git commit -m "Flutter lab: final"
git push
```

Common Pitfalls

- `Undefined class 'Timer'`: the import `dart:async` is missing.
- Forgetting `super.initState()` first or `super.dispose()` last.
- `setState` forgotten: the variable changes but the screen does not (Step 7).
- A key written as `Key("x")` with a different spelling, or missing `const`: the checker will not find the widget.
- The same key used on two widgets (for example `progress` still in `HomePage` and in `Tally`).
- Calling `Timer.periodic` inside `build`: it starts a new timer on every rebuild.
- `setState() called after dispose()`: the timer was not cancelled.
- Run `flutter pub get` if VS Code shows red imports right after `flutter create`.

Marking Scheme

Section	Marks	What is checked
Steps 1 to 10 (8 marks each)	80	The app passes the automatic test for the step with your own seed (4); the code uses the required feature correctly (3); the step was pushed by its checkpoint (1)
Final project	10	`flutter analyze` is clean, the code is formatted, `screenshot.png` is included, A1 to A4 are correct

Section	Marks	What is checked
Reflection	10	B1 to B3 and the Think comments
Total	**100**	